

Decision Tree Classification – Airline Customer Satisfaction

Step 1: Import Libraries

```
In [1]: 1 # Data manipulation
2 import pandas as pd
3 import numpy as np
4
5 # Visualization
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 # Machine Learning
10 from sklearn.model_selection import train_test_split, GridSearchCV
11 from sklearn.tree import DecisionTreeClassifier, plot_tree
12 from sklearn.preprocessing import LabelEncoder
13 from sklearn.metrics import (
14     confusion_matrix,
15     ConfusionMatrixDisplay,
16     classification_report,
17     accuracy_score,
18     f1_score
19 )
20 from sklearn.linear_model import LogisticRegression
21
22 # Ignore warnings
23 import warnings
24 warnings.filterwarnings("ignore")
```

Step 2: Load & Explore the Dataset

```
In [3]: 1 file_path = "C:\\Users\\DajahV01\\Desktop\\DHF Assignment\\28. Decision Tree Model Evaluation"
```

```
In [4]: 1 df = pd.read_csv(file_path)
        2
        3 # Display first five rows
        4 df.head()
```

Out[4]:

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	...	C
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	0	2	...	su
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	0	3	...	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	0	3	...	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	0	3	...	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	0	3	...	

5 rows × 22 columns



```
In [6]: 1 # Dataset dimensions
        2 print(df.shape)
        3
```

(129880, 22)



```
In [7]: 1 # Data types
        2 df.info()
        3
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 129880 entries, 0 to 129879
Data columns (total 22 columns):
#   Column                                          Non-Null Count  Dtype
---  -
0   satisfaction                                  129880 non-null  object
1   Customer Type                                129880 non-null  object
2   Age                                            129880 non-null  int64
3   Type of Travel                               129880 non-null  object
4   Class                                          129880 non-null  object
5   Flight Distance                              129880 non-null  int64
6   Seat comfort                                 129880 non-null  int64
7   Departure/Arrival time convenient            129880 non-null  int64
8   Food and drink                               129880 non-null  int64
9   Gate location                                129880 non-null  int64
10  Inflight wifi service                        129880 non-null  int64
11  Inflight entertainment                      129880 non-null  int64
12  Online support                               129880 non-null  int64
13  Ease of Online booking                      129880 non-null  int64
14  On-board service                            129880 non-null  int64
15  Leg room service                            129880 non-null  int64
16  Baggage handling                            129880 non-null  int64
17  Checkin service                             129880 non-null  int64
18  Cleanliness                                  129880 non-null  int64
19  Online boarding                             129880 non-null  int64
20  Departure Delay in Minutes                  129880 non-null  int64
21  Arrival Delay in Minutes                    129487 non-null  float64
dtypes: float64(1), int64(17), object(4)
memory usage: 21.8+ MB
```

```
In [8]: 1 # Summary statistics
        2 df.describe()
        3
        4
```

Out[8]:

	Age	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	In
count	129880.000000	129880.000000	129880.000000	129880.000000	129880.000000	129880.000000	129880.000000
mean	39.427957	1981.409055	2.838597	2.990645	2.851994	2.990422	2.990422
std	15.119360	1027.115606	1.392983	1.527224	1.443729	1.305970	1.305970
min	7.000000	50.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	27.000000	1359.000000	2.000000	2.000000	2.000000	2.000000	2.000000
50%	40.000000	1925.000000	3.000000	3.000000	3.000000	3.000000	3.000000
75%	51.000000	2544.000000	4.000000	4.000000	4.000000	4.000000	4.000000
max	85.000000	6951.000000	5.000000	5.000000	5.000000	5.000000	5.000000

```
In [9]: 1 # Missing values
        2 df.isnull().sum()
```

```
Out[9]: satisfaction                                0
Customer Type                                      0
Age                                                  0
Type of Travel                                      0
Class                                                0
Flight Distance                                     0
Seat comfort                                         0
Departure/Arrival time convenient                  0
Food and drink                                      0
Gate location                                       0
Inflight wifi service                              0
Inflight entertainment                             0
Online support                                      0
Ease of Online booking                             0
On-board service                                    0
Leg room service                                    0
Baggage handling                                    0
Checkin service                                     0
Cleanliness                                         0
Online boarding                                     0
Departure Delay in Minutes                          0
Arrival Delay in Minutes                           393
dtype: int64
```

Step 3: Handle Missing Values

As seen above one column has 393 missing values.

```
In [13]: 1 # Fill missing values with median of the column
        2
        3 df["Arrival Delay in Minutes"] = df["Arrival Delay in Minutes"].fillna(
        4     df["Arrival Delay in Minutes"].median()
        5 )
        6
```

```
In [14]: 1 # Verify
         2 df.isnull().sum()
```

```
Out[14]: satisfaction      0
Customer Type             0
Age                       0
Type of Travel            0
Class                     0
Flight Distance           0
Seat comfort              0
Departure/Arrival time convenient 0
Food and drink            0
Gate location             0
Inflight wifi service     0
Inflight entertainment    0
Online support            0
Ease of Online booking    0
On-board service          0
Leg room service          0
Baggage handling          0
Checkin service           0
Cleanliness               0
Online boarding           0
Departure Delay in Minutes 0
Arrival Delay in Minutes  0
dtype: int64
```

Step 4: Encode Categorical Variables

Decision Trees require numerical inputs.

```
In [15]: 1 encoder = LabelEncoder()
2         categorical_cols = df.select_dtypes(include="object").columns
3         for col in categorical_cols:
4             df[col] = encoder.fit_transform(df[col])
5         df.head()
```

Out[15]:

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	...	Online support
0	1	0	65	1	1	265	0	0	0	2	...	2
1	1	0	47	1	0	2464	0	0	0	3	...	2
2	1	0	15	1	1	2138	0	0	0	3	...	2
3	1	0	60	1	1	623	0	0	0	3	...	3
4	1	0	70	1	1	354	0	0	0	3	...	4

5 rows × 22 columns

Step 5: Define Features and Target

```
In [16]: 1 X = df.drop("satisfaction", axis=1)
2         y = df["satisfaction"]
3
4         print(X.shape)
5         print(y.shape)
```

(129880, 21)
(129880,)

Step 6: Train-Test Split

```
In [18]: 1 X_train, X_test, y_train, y_test = train_test_split(
2         X,
3         y,
4         test_size = 0.2,
5         random_state = 42,
6         stratify = y
7     )
```

Step 7: Hyperparameter Tuning with GridSearchCV

Search for the best Decision Tree.

```
In [19]: 1 param_grid = {
2         "max_depth": [3, 5, 7, 10, None],
3         "min_samples_split": [2, 5, 10],
4         "min_samples_leaf": [1, 2, 4]
5     }
6
7     grid_search = GridSearchCV(
8         DecisionTreeClassifier(random_state = 42),
9         param_grid=param_grid,
10        cv=5,
11        scoring = "f1",
12        n_jobs = -1
13    )
14
15    grid_search.fit(X_train, y_train)
16
17    print("Best Parameters:")
18    print(grid_search.best_params_)
```

Best Parameters:
{'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 10}

Step 8: Train the Final Decision Tree

```
In [20]: 1 best_tree = grid_search.best_estimator_  
         2  
         3 best_tree.fit(X_train, y_train)
```

```
Out[20]: ▼ DecisionTreeClassifier  
DecisionTreeClassifier(min_samples_split=10, random_state=42) ▴
```

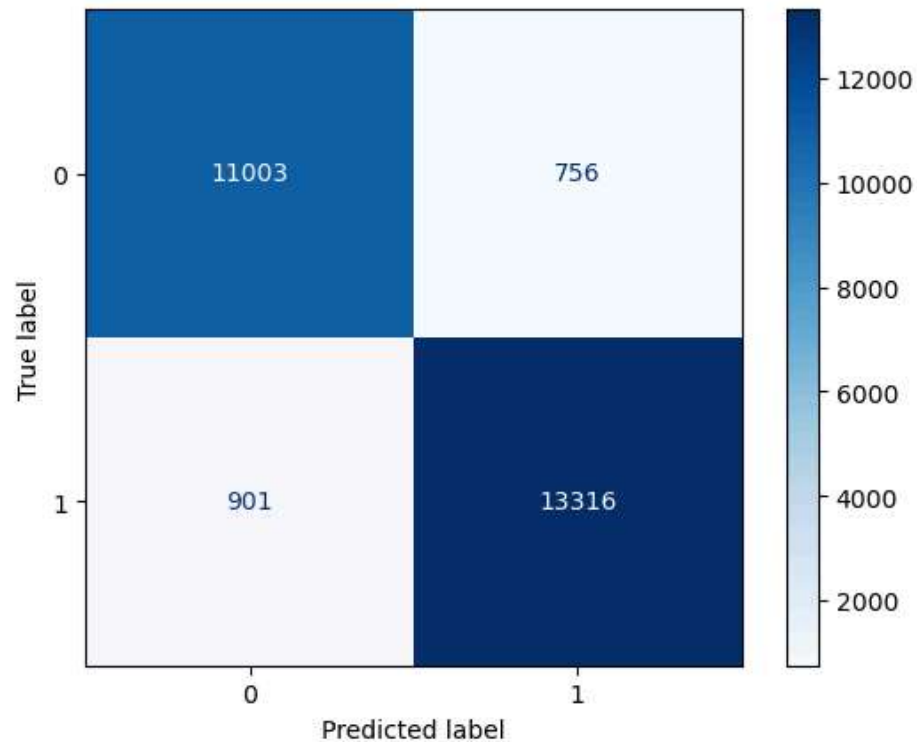
Step 9: Predictions

```
In [21]: 1 y_pred = best_tree.predict(X_test)
```

Step 10: Evaluate the Model

Confusion Matrix

```
In [23]: 1 cm = confusion_matrix(y_test, y_pred)
          2
          3 disp = ConfusionMatrixDisplay(cm)
          4
          5 disp.plot(cmap = "Blues")
          6 plt.show()
```



Accuracy

```
In [24]: 1 accuracy = accuracy_score(y_test, y_pred)
          2
          3 print("Accuracy:", accuracy)
```

Accuracy: 0.936210348013551

F1 Score

Assuming

- Satisfied = 1

```
In [25]: 1 f1 = f1_score(y_test, y_pred, pos_label=1)
          2
          3 print("F1 Score:", f1)
```

F1 Score: 0.9414259959701652

Classification Report

```
In [26]: 1 print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.92	0.94	0.93	11759
1	0.95	0.94	0.94	14217
accuracy			0.94	25976
macro avg	0.94	0.94	0.94	25976
weighted avg	0.94	0.94	0.94	25976

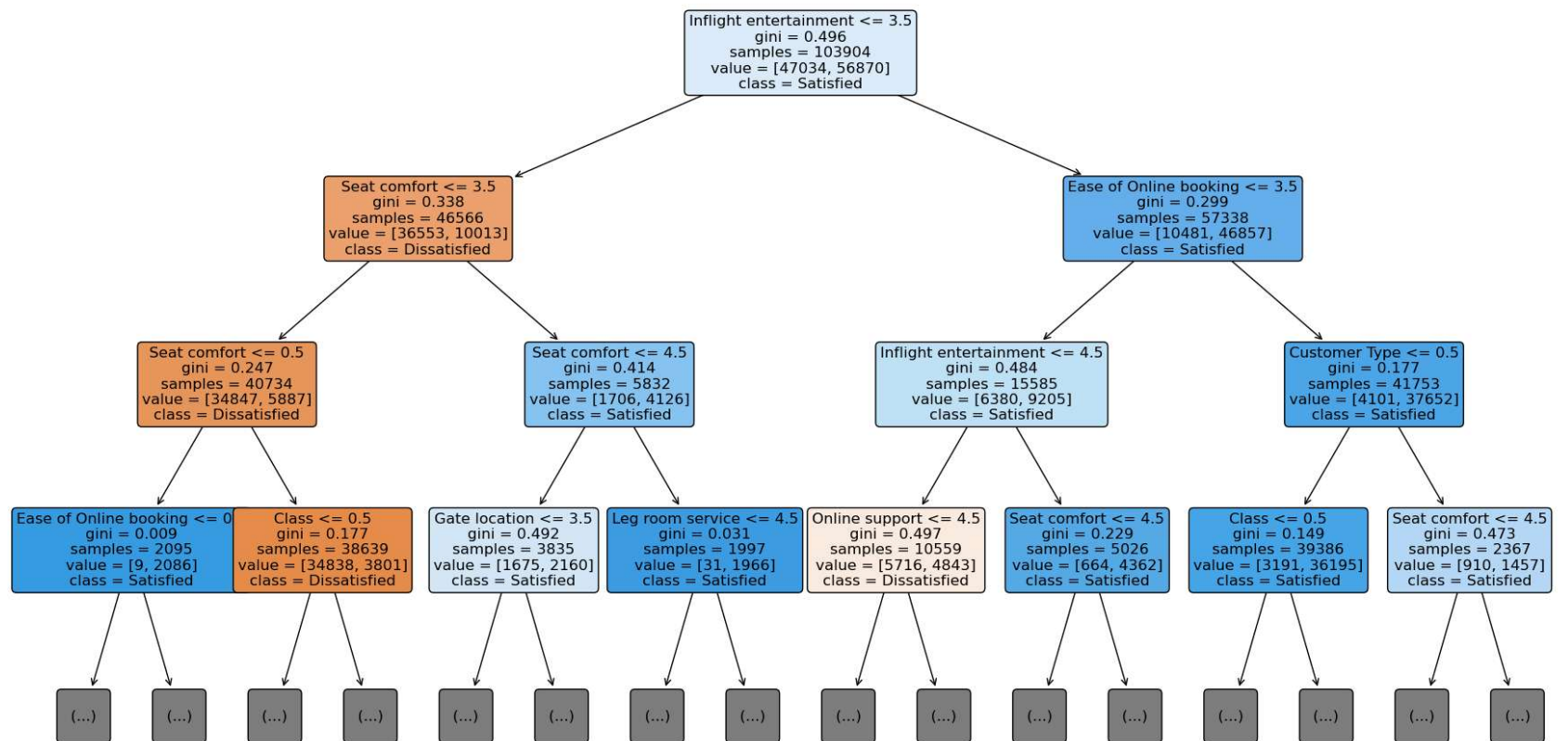
Step 11: Visualize the Decision Tree

To keep the visualization readable, limit the displayed depth.

```

In [31]: 1 plt.figure(figsize=(22,12))
          2
          3 plot_tree(
          4     best_tree,
          5     feature_names = X.columns,
          6     class_names = ["Dissatisfied", "Satisfied"],
          7     filled = True,
          8     rounded = True,
          9     fontsize = 12,
         10     max_depth = 3
         11 )
         12
         13 plt.show()

```



Step 12: Feature Importance

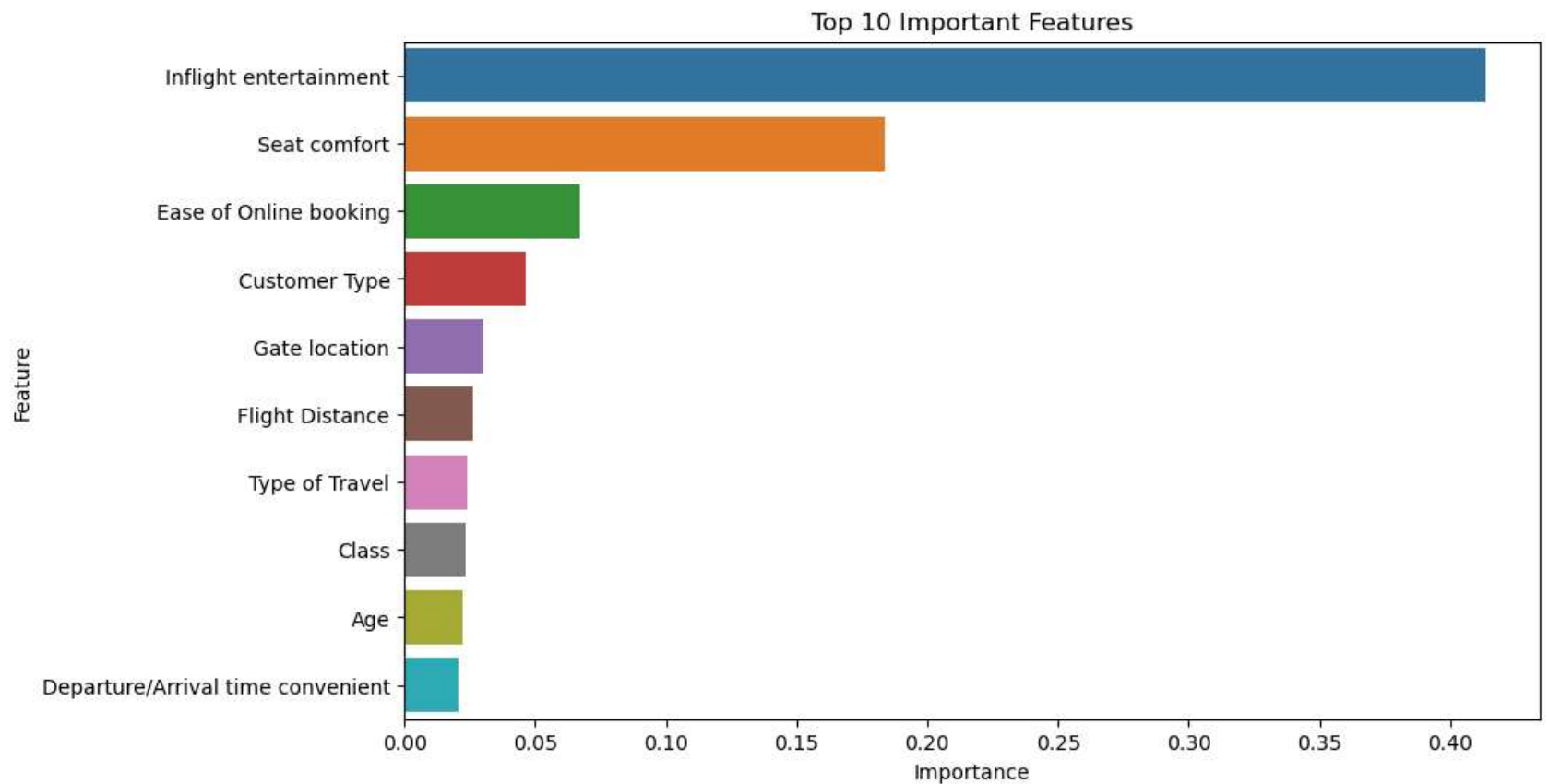
```
In [33]: 1 importance = pd.DataFrame({
2         "Feature": X.columns,
3         "Importance": best_tree.feature_importances_
4     })
5
6 importance = importance.sort_values(
7     by = "Importance",
8     ascending = False
9 )
10
11 importance.head(10)
```

Out[33]:

	Feature	Importance
10	Inflight entertainment	0.413691
5	Seat comfort	0.183479
12	Ease of Online booking	0.067222
0	Customer Type	0.046357
8	Gate location	0.030255
4	Flight Distance	0.026399
2	Type of Travel	0.024098
3	Class	0.023305
1	Age	0.022237
6	Departure/Arrival time convenient	0.020454

Visualization

```
In [35]: 1 plt.figure(figsize=(10,6))
2
3 sns.barplot(
4     data=importance.head(10),
5     x="Importance",
6     y="Feature"
7 )
8
9 plt.title("Top 10 Important Features")
10 plt.show()
```



Step 13: Compare with Logistic Regression

```
In [36]: 1 log_model = LogisticRegression(max_iter=1000)
          2
          3 log_model.fit(X_train, y_train)
          4
          5 log_pred = log_model.predict(X_test)
```

Evaluate.

```
In [37]: 1 print("Decision Tree")
          2
          3 print(classification_report(y_test, y_pred))
          4
          5 print()
          6
          7 print("Logistic Regression")
          8
          9 print(classification_report(y_test, log_pred))
```

Decision Tree

	precision	recall	f1-score	support
0	0.92	0.94	0.93	11759
1	0.95	0.94	0.94	14217
accuracy			0.94	25976
macro avg	0.94	0.94	0.94	25976
weighted avg	0.94	0.94	0.94	25976

Logistic Regression

	precision	recall	f1-score	support
0	0.81	0.78	0.79	11759
1	0.82	0.84	0.83	14217
accuracy			0.82	25976
macro avg	0.81	0.81	0.81	25976
weighted avg	0.82	0.82	0.82	25976

Step 14: Interpretation

Decision Tree Algorithm

Advantages

- Easy to interpret
- Can be visualized
- Captures nonlinear relationships
- Provides feature importance

Disadvantages

- Can overfit without pruning
- Sensitive to small changes in data

Logistic Regression Algorithm

Advantages

- Simple and fast
- Performs well on linearly separable data
- Stable predictions

Disadvantages

- Assumes a linear relationship
- Less interpretable for complex interactions

Step 15: Business Insights

Example insights based on the model output:

- Inflight entertainment is one of the strongest predictors of customer satisfaction, suggesting that enhancing onboard entertainment options could significantly improve passenger experience.
- Online support and Ease of Online booking are influential factors, indicating that investments in digital customer service and booking platforms may increase satisfaction.

- Seat comfort consistently ranks among the most important features, highlighting the value of cabin comfort improvements.
- Leg room service also contributes meaningfully, especially for long-haul travelers.
- Departure and Arrival delays have lower importance than service-quality features, implying that while punctuality matters, the overall onboard experience has a stronger impact on satisfaction.